

NYC Borough Location Classification

Business Overview and Technical Summary



Executive Summary

This process creates a clean, report-ready file that helps business users determine whether a client address falls inside one of the five New York City boroughs. It compares address latitude and longitude values from SQL Server against official NYC borough boundary geometry from a GeoJSON file, then assigns each address an inside/outside status and, when applicable, the matching borough name.

Business Purpose

The goal of this work is to support location verification for call-law and operational review. Instead of manually checking whether an address is located within the New York City boroughs, the process uses geographic boundary data to classify addresses consistently and prepares the results for Power BI reporting.

Inputs Used

Input	Purpose	Notes from the Python Code
SQL Server address data	Provides AddressKey, latitude, and longitude values for classification.	The script defaults to server cinshrsqlo5 and table dbo.Address_vw. The database is supplied when the script is run.
NYC borough GeoJSON	Provides the official borough boundary shapes used for the spatial comparison.	The GeoJSON must include a BoroName field.
Bounding box filters	Narrows the SQL extract to the NYC-area latitude and longitude range before classification.	Default filters are latitude 40.45 to 41.10 and longitude -74.40 to -73.40.

How the Process Works

1. Read borough boundaries: The script loads the NYC borough GeoJSON and confirms that the file contains the BoroName field needed to label each matching address.

2. Connect to SQL Server: The script connects to SQL Server using pyodbc and retrieves address records in batches, which helps the process handle large data volumes without loading everything at once.

- 3. Filter to the NYC area:** Only records with non-null latitude and longitude values inside the configured NYC-area bounding box are pulled into each batch.
- 4. Convert addresses into map points:** Each valid address is converted into a geographic point using longitude and latitude.
- 5. Match points to borough polygons:** A spatial join compares each address point to the borough boundary polygons. If the point intersects with a borough boundary area, the record is labeled inside and receives the borough name.
- 6. Write report-ready output:** The results are appended to a CSV containing AddressKey, latitude, longitude, borough, and inside_outside status. This file can then be loaded into Power BI.

Output Produced

The generated CSV is designed to be simple and Power BI friendly. Each row represents one address record that was evaluated by the script.

Output Field	Business Meaning
AddressKey	Unique address identifier from the source data.
Latitude_Decimal	Latitude used to place the address on the map.
Longitude_Decimal	Longitude used to place the address on the map.
borough	NYC borough name when the point falls inside a borough boundary.
inside_outside	Classification status: inside, outside, or invalid_coords.

Power BI Usage

After the Python script creates the CSV output, the file can be loaded into Power BI as a separate reporting step. In Power BI, the address points can be plotted on an Azure Maps visual, and the NYC borough GeoJSON can be added as a boundary layer so users can compare address locations against borough coverage. The inside/outside flag and borough field then give users a quick way to filter, validate, and summarize location results.